


```
no:
[books] (1355 docs)
```

Solid line = word present (yes), dashed line = word absent (no).

Method 1 produces a degenerate chain — each split peels off a small group of atheism docs from one side, leaving nearly all docs in the other child. IG values are all close to 0.5 because the (1/2, 1/2) weighting inflates gains from lopsided splits.

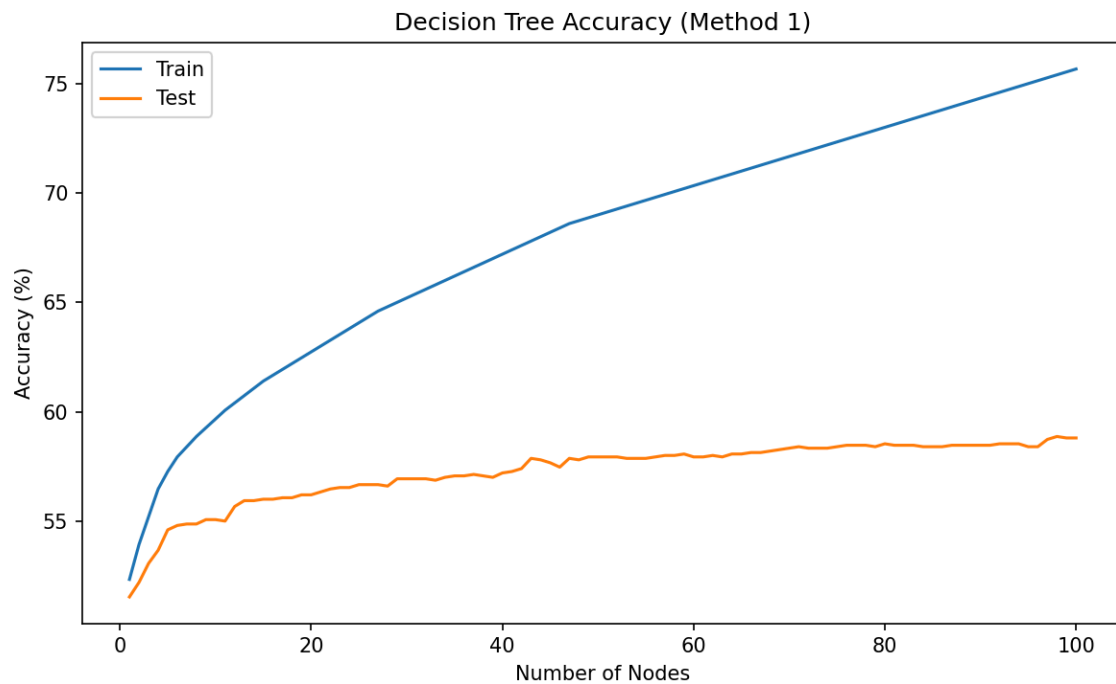
Method 2 (fraction-weighted IG)

```
book (IG=0.077019)
  yes:
    bible (IG=0.115358)
      yes:
        [atheism] (4 docs)
      no:
        call (IG=0.069459)
          yes:
            [atheism] (2 docs)
          no:
            sent (IG=0.084224)
              yes:
                [atheism] (2 docs)
              no:
                controlling (IG=0.058758)
                  yes:
                    [atheism] (1 docs)
                  no:
                    [books] (146 docs)
        no:
          books (IG=0.059926)
            yes:
              sure (IG=0.097695)
                yes:
                  soon (IG=0.918296)
                    yes:
                      [books] (1 docs)
                    no:
                      [atheism] (2 docs)
                no:
                  spirit (IG=0.096945)
                    yes:
                      [atheism] (1 docs)
                    no:
                      [books] (79 docs)
            no:
              religion (IG=0.035673)
                yes:
                  [atheism] (65 docs)
                no:
                  [atheism] (1197 docs)
```

Method 2 produces a proper tree structure — root splits on "book" into two meaningful subtrees. IG values are smaller but more realistic. The fraction weighting penalizes lopsided splits correctly.

Accuracy Curves

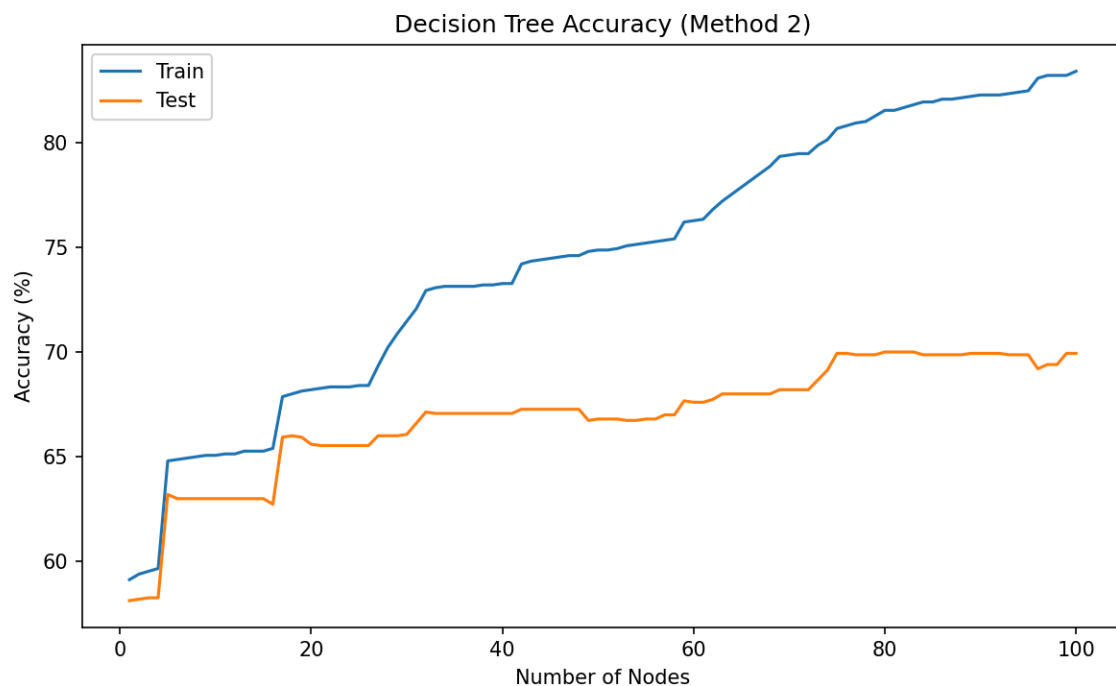
Method 1



Final accuracy at 100 nodes — Train: 75.67%, Test: 58.80%.

Train accuracy keeps climbing, test accuracy plateaus around 57-59% after ~15 nodes. The growing gap = overfitting, consistent with Method 1 picking features that separate tiny groups.

Method 2



Final accuracy at 100 nodes — Train: 83.40%, Test: 69.93%.

Both train and test accuracy are higher than Method 1. Test accuracy reaches ~70% and stays more stable — fraction-weighted IG picks more informative splits.

Question 2 [65pts]: Neural Networks

Implementation

Code in `neural_net.py` and `operations.py`. All public tests pass (10/10).

K-Fold Cross Validation (k=5)

Network architecture: [16, 1] — one hidden layer of 16 neurons with Sigmoid activation, output layer of 1 neuron with Identity activation.

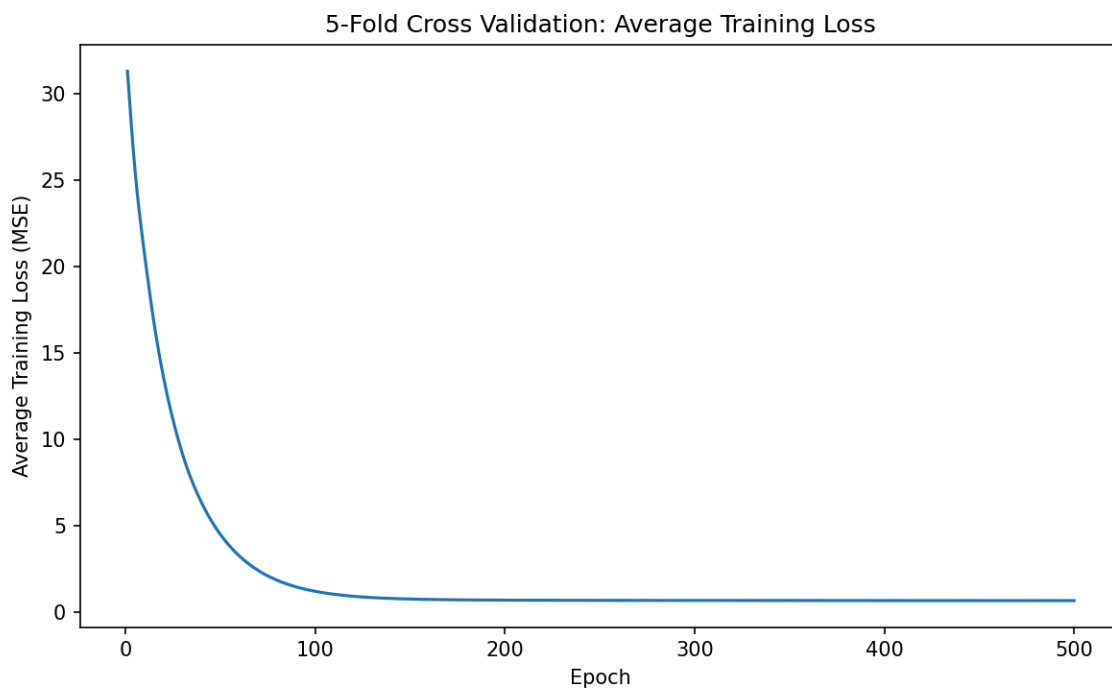
Hyperparameters: learning rate $\eta = 0.001$, 500 epochs.

Results:

Fold	Validation MAE
1	0.6546
2	0.6574
3	0.6925
4	0.6980
5	0.5985

Average MAE: 0.6602 ± 0.0356

Average Training Loss



Loss drops from ~31 to ~0.6 in the first 150 epochs, then stabilizes. Standard convergence behavior for batch gradient descent with a small learning rate.

Bonus Question [40pts]: Training with PyTorch

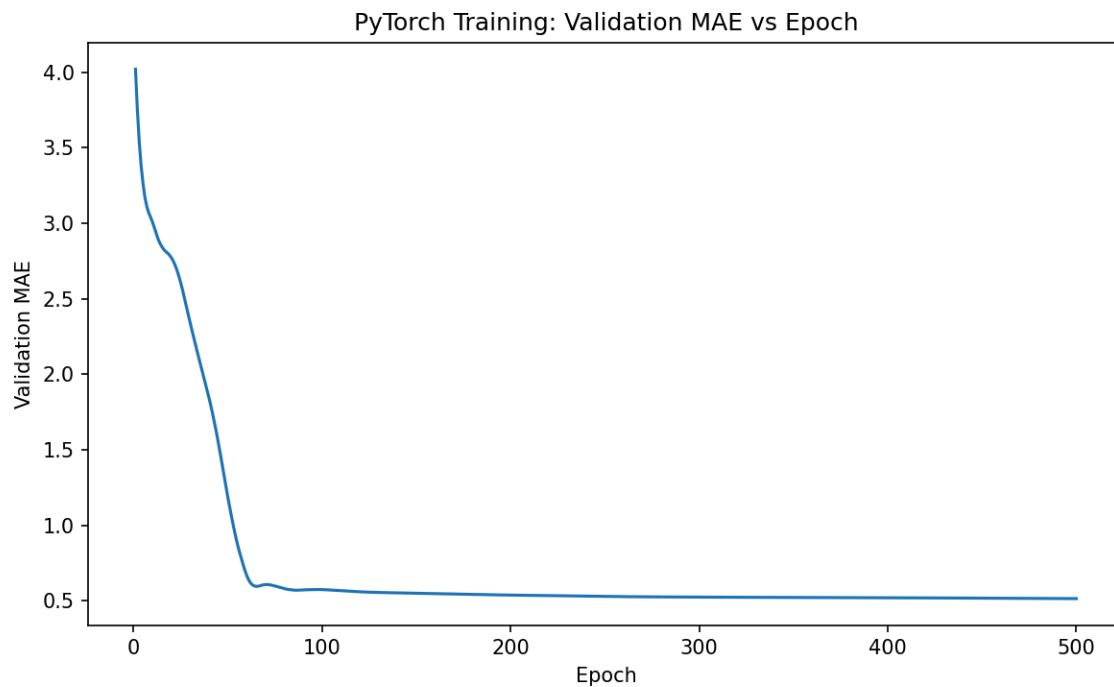
Code in `train_model_pytorch.py`.

Network architecture: `Linear(11, 64) → ReLU → Linear(64, 32) → ReLU → Linear(32, 1)`.

Setup: 90/10 train/validation split (seed=42), MSELoss, Adam optimizer, lr=0.001, 500 epochs.

Final validation MAE: 0.5147

Validation MAE vs Epoch



MAE drops from ~4.0 to ~0.6 in the first 80 epochs, then gradually decreases to 0.51. The deeper network + Adam optimizer achieves lower MAE than the from-scratch implementation (0.66 with k-fold).